Structured Mesh Generation

Once upon a time, I assigned a Laplace problem in an ell-shaped region using a mesh that looked something like the one shown in Figure 1. This is not necessarily an optimal mesh for this problem. The fastest variation in the solution is found in the corner at $(i, j) = (4, 4)$. Meanwhile, there is nearly no variation of the solution at $(8, 8)$, but the resolution is the same. We're going to use this geometry as an example while exploring mesh generation techniques.

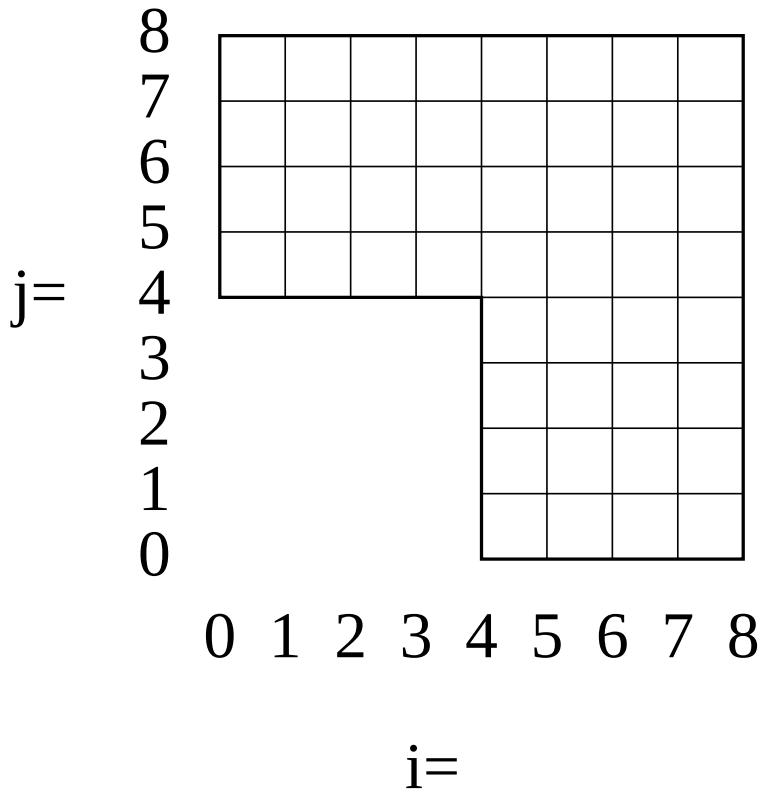


Figure 1: Ell-shaped test region for mesh generation with simple mesh.

1 Algebraic Interpolation Methods

Suppose we want to generate a mesh with lines running (roughly) radially (j-direction) and tangentially (i-direction) for this problem. We might discretize the boundary as shown in

Figure 2. The obvious thing to do is to interpolate linearly between the inner and outer boundaries. For a 32×16 mesh, this gives the mesh in Figure 3.

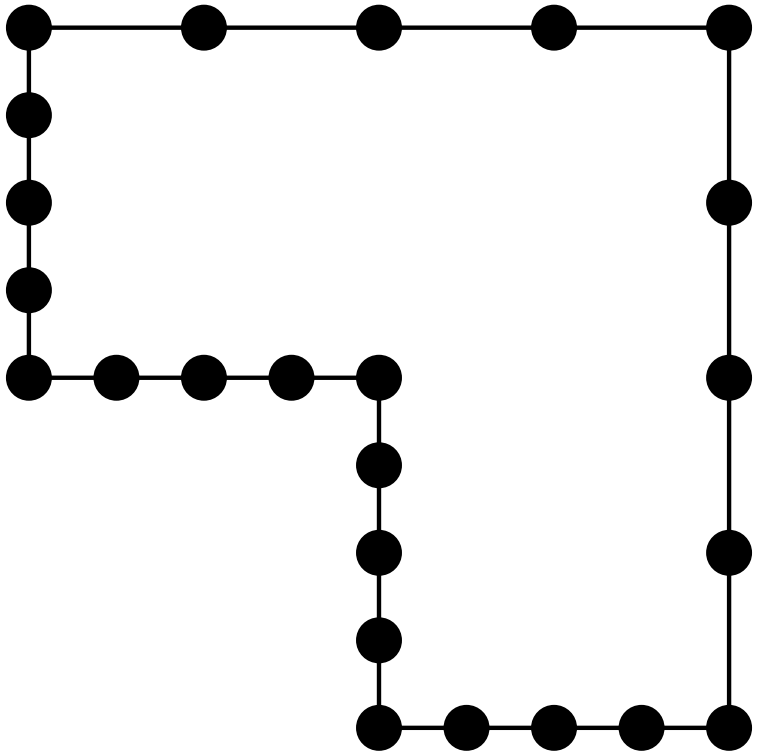


Figure 2: Sample discretization for 8x4 mesh in ell-shape.

The discontinuity in the mesh along the diagonal is a feature of this approach to mesh generation; boundary discontinuities are propagated into the mesh. This is undesirable from a computational point of view because accuracy degrades at slope discontinuities in a mesh. See the Generalized Coordinates handout for more info.

It isn't hard to convince yourself that interpolating the other direction, "tangentially", fails for this case. The question is, what to do about this in general?

1.1 Transfinite Interpolation

For algebraic mesh generation problems, we know in advance the boundary discretization. That is, we know x and y along the lines $i = 0$, $i = i_{max}$, $j = 0$, and $j = j_{max}$. The

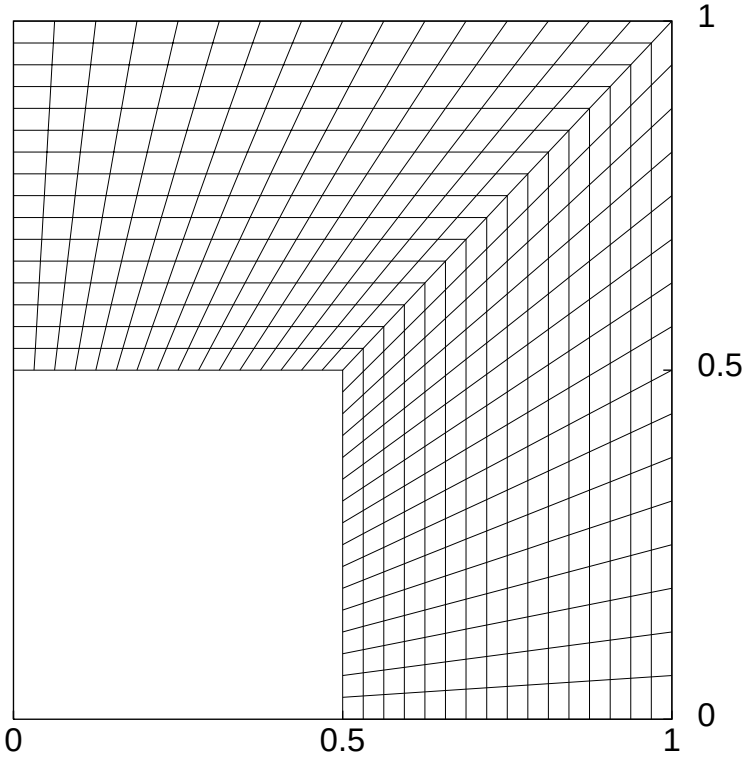


Figure 3: 16x32 mesh generated using interpolation.

interpolation scheme used for the previous mesh used only part of this data:

$$\begin{aligned} x_{i,j} &= x_{i,j_{max}} \frac{j}{j_{max}} + x_{i,0} \left(1 - \frac{j}{j_{max}}\right) \\ y_{i,j} &= y_{i,j_{max}} \frac{j}{j_{max}} + y_{i,0} \left(1 - \frac{j}{j_{max}}\right) \end{aligned} \tag{1}$$

This interpolation exactly reproduces the $j = 0$ and $j = j_{max}$ boundaries. But suppose that the $i = 0$ and $i = i_{max}$ boundaries are not straight. Then, along $i = 0$ the error (between the interpolated values and the correct boundary values) would be

$$\begin{aligned} \Delta x_{0,j} &= x_{0,j} - x_{0,j_{max}} \frac{j}{j_{max}} - x_{0,0} \left(1 - \frac{j}{j_{max}}\right) \\ \Delta y_{0,j} &= y_{0,j} - y_{0,j_{max}} \frac{j}{j_{max}} - y_{0,0} \left(1 - \frac{j}{j_{max}}\right) \end{aligned} \tag{2}$$

A similar expression can be written for the error at $i = i_{max}$. Note that this error is zero at the corners. We can interpolate this error between the $i = 0$ and $i = i_{max}$ boundaries:

$$\begin{aligned} \Delta x_{i,j} &= \Delta x_{0,j}\left(1 - \frac{i}{i_{max}}\right) + \Delta x_{i_{max},j}\frac{i}{i_{max}} \\ \Delta y_{i,j} &= \Delta y_{0,j}\left(1 - \frac{i}{i_{max}}\right) + \Delta y_{i_{max},j}\frac{i}{i_{max}} \end{aligned} \tag{3}$$

Because the error is zero at the corners, when we interpolate in the i-direction, we get Δx and Δy both identically zero along the $j = 0$ and $j = j_{max}$ boundaries. This means that we can add the results of Equations 1 and 3 without causing geometric errors along the $j = 0$ and $j = j_{max}$ boundaries. At the same time, we will fix the geometric errors along the $i = 0$ and $i = i_{max}$ boundaries. Formally, the sum of Equations 1 and 3 is

$$\begin{aligned} x_{i,j} &= x_{i,j_{max}}\frac{j}{j_{max}} + x_{i,0}\left(1 - \frac{j}{j_{max}}\right) + x_{i_{max},j}\frac{i}{i_{max}} + x_{0,j}\left(1 - \frac{i}{i_{max}}\right) \\ &\quad - x_{i_{max},j_{max}}\frac{i}{i_{max}}\frac{j}{j_{max}} - x_{i_{max},0}\frac{i}{i_{max}}\left(1 - \frac{j}{j_{max}}\right) \\ &\quad - x_{0,j_{max}}\left(1 - \frac{i}{i_{max}}\right)\frac{j}{j_{max}} - x_{0,0}\left(1 - \frac{i}{i_{max}}\right)\left(1 - \frac{j}{j_{max}}\right) \\ y_{i,j} &= y_{i,j_{max}}\frac{j}{j_{max}} + y_{i,0}\left(1 - \frac{j}{j_{max}}\right) + y_{i_{max},j}\frac{i}{i_{max}} + y_{0,j}\left(1 - \frac{i}{i_{max}}\right) \\ &\quad - y_{i_{max},j_{max}}\frac{i}{i_{max}}\frac{j}{j_{max}} - y_{i_{max},0}\frac{i}{i_{max}}\left(1 - \frac{j}{j_{max}}\right) \\ &\quad - y_{0,j_{max}}\left(1 - \frac{i}{i_{max}}\right)\frac{j}{j_{max}} - y_{0,0}\left(1 - \frac{i}{i_{max}}\right)\left(1 - \frac{j}{j_{max}}\right) \end{aligned}$$

While that's very nice, in practice it's much more useful to sum the results of Equations 1 and 3. This technique is known as *transfinite interpolation*. Figure 4 shows an example of the kind of mesh you can get from transfinite interpolation. However, this can break down for some boundary shapes. For example, if the curve in the lower boundary here is made larger, we get an invalid mesh, as seen in Figure 5. Breakdown can also occur in the interior of the mesh, with cells folding back on each other.

2 Elliptic Mesh Generation

Let's take a step back and examine what we know so far.

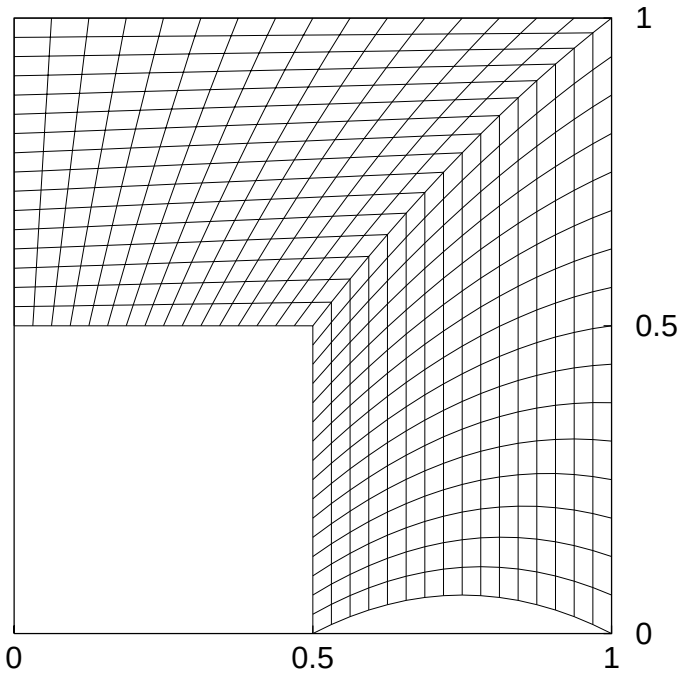


Figure 4: Mesh for perturbed ell-shape generated using transfinite interpolation.

1. We know that transfinite interpolation is not the perfect approach to structured mesh generation; it is subject to failures for some geometries and produces meshes which have discontinuities in mesh line angles (and therefore mesh metrics and Jacobians).

2. We have a boundary discretization such that only one index (i or j, corresponding to ξ and η, respectively) varies along each part of the boundary. Furthermore, the variation of i and j (and therefore ξ and η) is monotone along the boundary. That is, successive boundary points in the physical space are numbered 0, 1, 2, ... i_{max} in order. This is a necessary but not sufficient condition for the mesh to be valid; a necessary *and* sufficient condition is that ξ and η have extrema only on the boundary of the domain. In practice, what this means is that you can't have points numbered from 0 to 16, say, on the boundary but 0 to 18 in the interior.

3. We know that solutions to Laplace's equation have extrema only on the boundary of the domain.

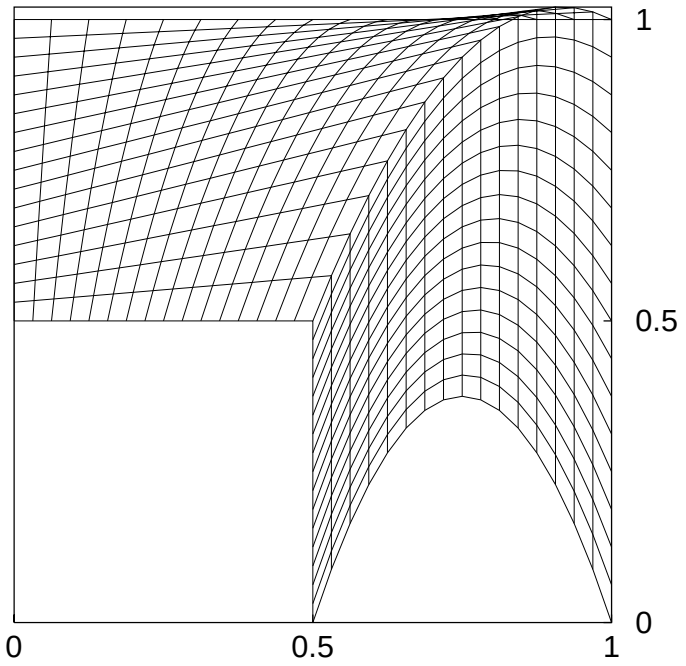


Figure 5: Mesh showing breakdown of transfinite interpolation.

Points 2 and 3 should make us think about using some sort of Laplacian-like mesh generator. Let's examine a more general elliptic problem instead, to give us some freedom to enforce mesh sizing, etc, later on.

$$\begin{aligned} \xi_{xx} + \xi_{yy} &= \left(\xi_x^2 + \xi_y^2\right) P \\ \eta_{xx} + \eta_{yy} &= \left(\eta_x^2 + \eta_y^2\right) Q \end{aligned} \tag{4}$$

The terms multiplying the sources P and Q are arc lengths in the physical space. Note that this is a finite-difference formulation, because we seek (ultimately) the values of x and y at the computational points in the (ξ, η)-plane where ξ and η take on integer values (i and j).

Unfortunately, we don't have any way to evaluate the derivatives in Equations 4, because we don't *know* where the mesh points are yet. We need to transform these equations into a set of equations containing derivatives of x and y in terms of ξ and η. While this transformation is not outrageously difficult, it isn't particularly relevant for us, and it is somewhat lengthy,

so we'll skip right to the result:

$$g_{22}\begin{pmatrix} x_{\xi\xi} \\ y_{\xi\xi} \end{pmatrix} + 2g_{12}\begin{pmatrix} x_{\xi\eta} \\ y_{\xi\eta} \end{pmatrix} + g_{11}\begin{pmatrix} x_{\eta\eta} \\ y_{\eta\eta} \end{pmatrix} = -\begin{pmatrix} g_{22}x_\xi P + g_{11}x_\eta Q \\ g_{22}y_\xi P + g_{11}y_\eta Q \end{pmatrix} \tag{5}$$

where

$$\begin{aligned} \vec{g}_1 &= \begin{pmatrix} x_\xi \\ y_\xi \end{pmatrix} \\ \vec{g}_2 &= \begin{pmatrix} x_\eta \\ y_\eta \end{pmatrix} \\ g_{11} &= \vec{g}_1 \cdot \vec{g}_1 = x_\xi^2 + y_\xi^2 \\ g_{12} &= \vec{g}_1 \cdot \vec{g}_2 = x_\xi x_\eta + y_\xi y_\eta \\ g_{22} &= \vec{g}_2 \cdot \vec{g}_2 = x_\eta^2 + y_\eta^2 \end{aligned}$$

This can be implemented by using centered finite differences. Assuming that $\Delta\xi = \Delta\eta = 1$,

$$\begin{aligned} (\cdot)_{\xi\xi} &= (\cdot)_{i+1,j} - 2(\cdot)_{i,j} + (\cdot)_{i-1,j} \\ (\cdot)_{\xi\eta} &= (\cdot)_{i+1,j+1} - (\cdot)_{i+1,j-1} - (\cdot)_{i-1,j+1} + (\cdot)_{i-1,j-1} \\ (\cdot)_{\eta\eta} &= (\cdot)_{i,j+1} - 2(\cdot)_{i,j} + (\cdot)_{i,j-1} \\ (\cdot)_{\xi} &= (\cdot)_{i+1,j} - (\cdot)_{i-1,j} \\ (\cdot)_{\eta} &= (\cdot)_{i,j+1} - (\cdot)_{i,j-1} \end{aligned} \tag{6}$$

The usual sorts of Poisson solution techniques can be used. The examples in this handout were all done using point Gauss-Seidel with SOR.

For the original ell-shaped region of Figure 1 with P and Q both equal to zero, this method gives the mesh shown in Figure 6. This mesh is much smoother than the transfinite interpolation mesh of Figure 3, and also has a natural concentration of points near the interior corner, where we know the solution of the Laplacian varies steeply. This isn't actually all that surprising, given that we're essentially solving the same equation for ξ and η as we would for T in the Laplace problem (with appropriate boundary conditions). To demonstrate what an elliptic generator does for less regular boundaries, the domains of Figures 4 and 5 are meshed with the same generator and shown in Figure 7.

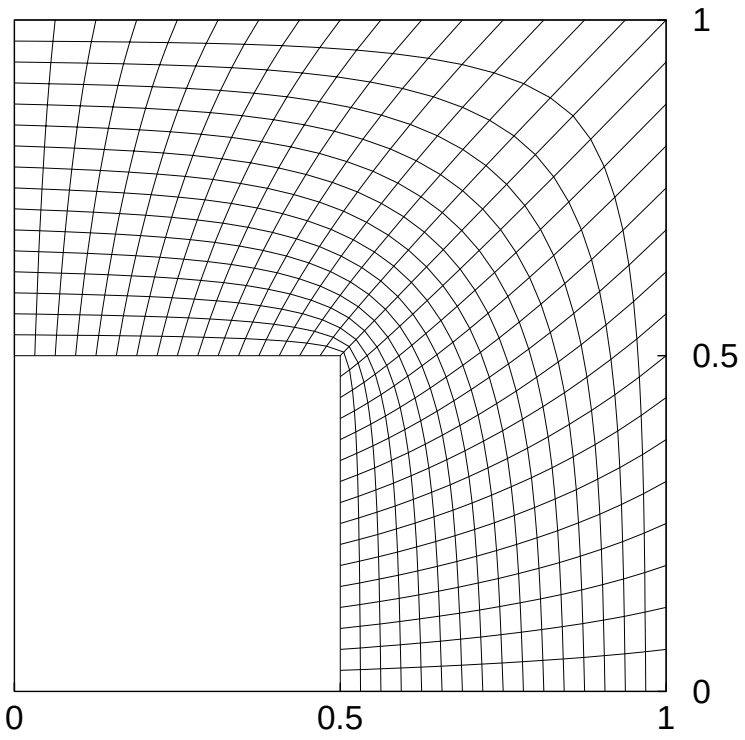


Figure 6: Mesh generated using elliptic system of Equations 5.

2.1 Source Terms for Point Density Control[1]

So far, so good. But suppose we want to have unequal boundary spacing, to concentrate points in some part of the domain? To take a concrete example, suppose that we are calculating something in a square and want to concentrate points near one corner. As part of this, we concentrate boundary points near that corner. Then we use an elliptic mesh generator with no source term to generate a mesh; this mesh appears on the left in Figure 8. While the mesh is smooth and nearly orthogonal, especially away from the boundaries, the point spacing is not at all what was specified on the boundary. What we were after looks like the right side of Figure 8.

To generate the mesh on the right of the figure, we need to ensure that Equation 5 is satisfied

[1]This section doesn't even begin to cover the variations and sophistication available in using source terms to modify point space and mesh orthogonality. See [1] for more details, and search the journal literature for recent developments.

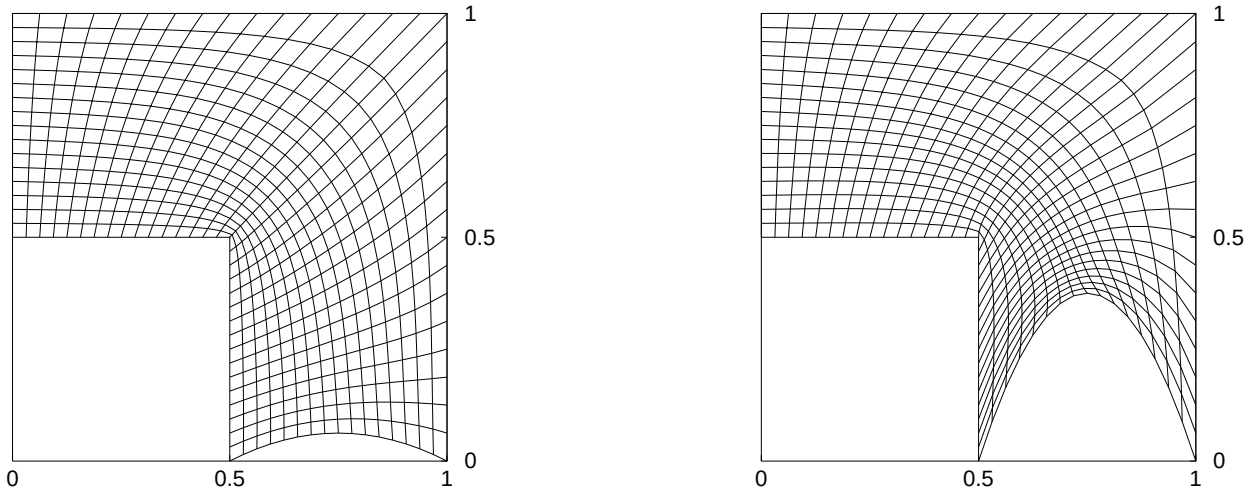


Figure 7: Meshes using elliptic generator for perturbed boundaries.

exactly for that mesh. For this simple case, $x = x(\xi)$ and $y = y(\eta)$, so Equation 5 reduces to

$$\begin{aligned} x_{\xi\xi} &= -x_\xi P \\ y_{\eta\eta} &= -y_\eta Q \end{aligned}$$

If we compute $P = -\frac{x_{\xi\xi}}{x_\xi}$ along the top and bottom boundaries, and $Q = -\frac{y_{\eta\eta}}{y_\eta}$ along the right and left boundaries, then we can interpolate both P and Q into the interior to get the correct source term for this simple case, recovering the mesh on the right side of Figure 8. For the more general case of non-coordinate aligned boundaries, we can use

$$\begin{aligned} P &= -\frac{x_\xi x_{\xi\xi} + y_\xi y_{\xi\xi}}{x_\xi^2 + y_\xi^2} \\ Q &= -\frac{x_\eta x_{\eta\eta} + y_\eta y_{\eta\eta}}{x_\eta^2 + y_\eta^2} \end{aligned}$$

along the boundaries. Neither of these fractions can ever involve division be zero, because the denominators are arclengths along the boundaries.

As a final example of mesh generation with source terms, suppose we want to attract mesh lines towards the diagonal in the mesh of Figure 9. From Equation 5, we can deduce that reducing (increasing) P will cause lines of constant ξ to move in the direction of decreasing (increasing) ξ, so to concentrate lines near $\xi = \xi^*$ requires P to be smaller for $\xi > \xi^*$ and larger for $\xi < \xi^*$ than for $\xi = \xi^*$. One functional form for this, and the one used for this

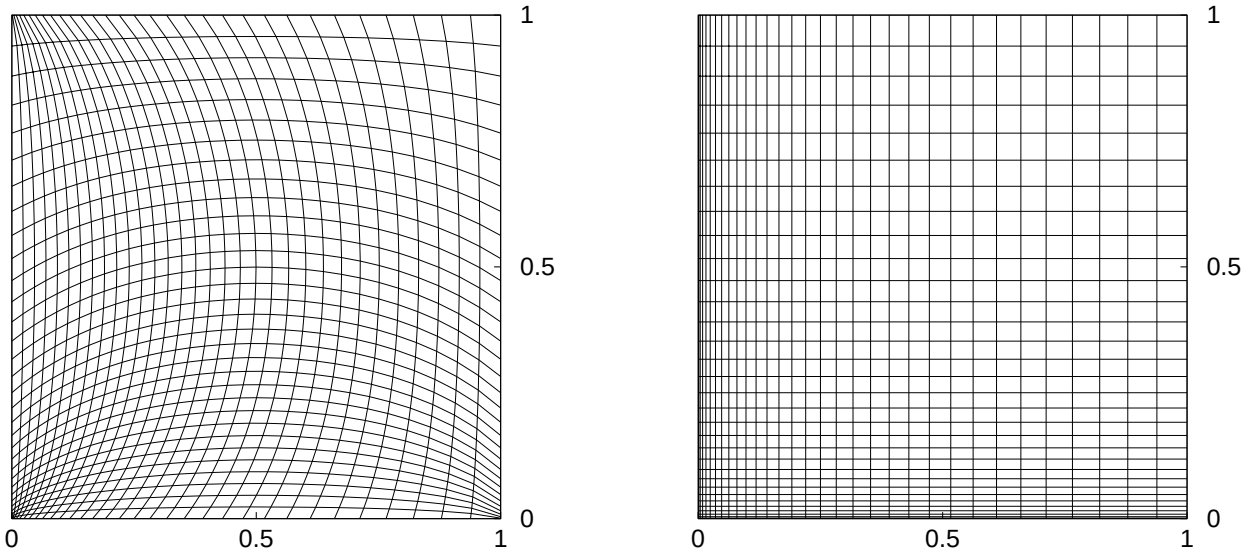


Figure 8: Absence of source term leads to poor point concentration (left). Correct source term gives desired result (right).

example, is

$$P = \frac{1}{4} sign\left(\xi - \xi^*\right) \exp\left(-\frac{1}{8} abs\left(\xi - \xi^*\right)\right)$$

The result is shown in Figure 10. Obviously, no effort was made to keep mesh lines orthogonal to the boundary for this example.

2.2 An Application Example: Mesh In the Bed of a Rotary Kiln

A student in 1998-99 had a research project involving particle flow in a rotary kiln, as shown very roughly in cross-section in Figure 11. The *bed,* or bottom, of the kiln is filled with lime particles (think of a mixture of sand and gravel). To model this flow, obviously the student needed a mesh for the bed. Alas, the region to be meshed has only two obvious logical sides, while a structured 2D mesh has four. This means that two extra "corners" must be inserted. Two possible locations for these are shown in the figure, one with $\mathbf{x}$'s and one with $\circ$'s. These corners will introduce some very oddly shaped cells, as we will see. The meshes that result from these divisions are shown in Figures 12 and 13. While I am certain that I prefer the elliptically-generated meshes to the transfinite interpolation meshes, I can't say offhand which of the elliptically-generated meshes I would prefer. Probably I would try both (and variations on them that give more appropriate mesh spacing, perhaps) and see what

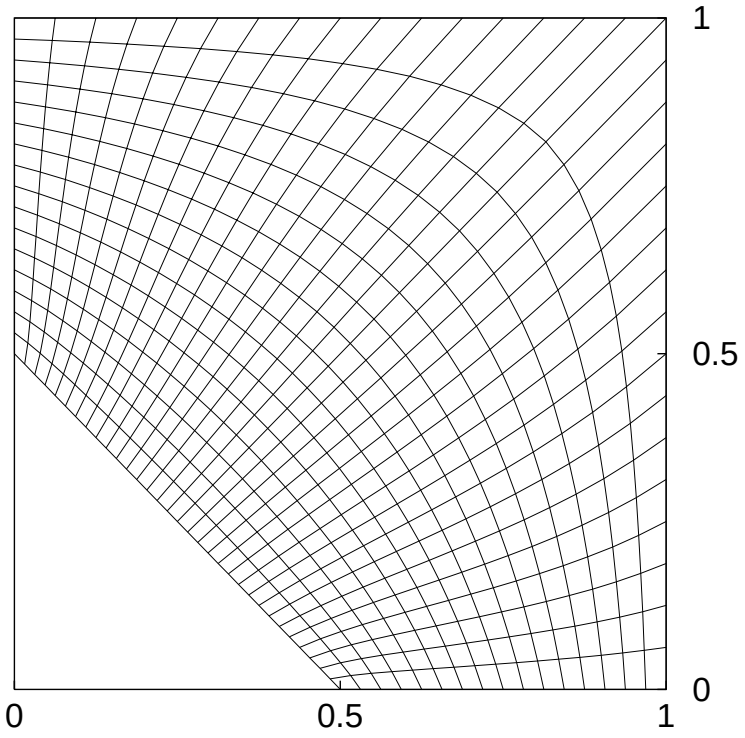


Figure 9: Mesh in pentagon with no source term.

the results looked like. Another possibility would be to have mesh singularities in the sharp corners; some flow solvers can handle this, while others can't.

3 Other Structured Mesh Generation Topics

These descriptions only touch the surface of these topics. If you're interested in knowing more, let me know, or go to your local library and do a literature search. All of these topics are discussed extensively in the literature.

3.1 Hyperbolic Mesh Generation

For external flow problems, meshes are often generated using hyperbolic mesh generators, which march out from the surface of a body to the farfield. The location of mesh points on the inner boundary and the angle of mesh lines at that boundary can be set as boundary

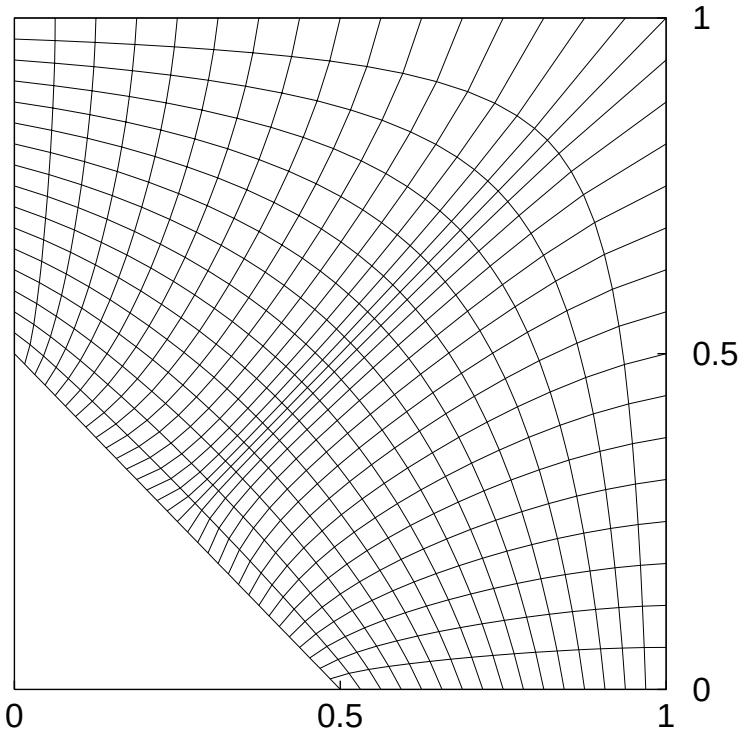


Figure 10: Mesh in pentagon with source term to concentrate mesh lines along $\xi = 16$.

conditions. The location of the outer boundary is determined during mesh generation. The mesh generator solves an equation which specifies mesh orthogonality,

$$x_\xi x_\eta + y_\xi y_\eta = 0$$

with, typically, a pre-specified cell volume:

$$x_\xi y_\eta - x_\eta y_\xi = V(\xi, \eta)$$

This system can be shown to be mathematically hyperbolic.

There are two significant advantages of hyperbolic mesh generation over elliptic:

1. Hyperbolically generated meshes are orthogonal.
2. Hyperbolic mesh generation is much faster than elliptic mesh generation.

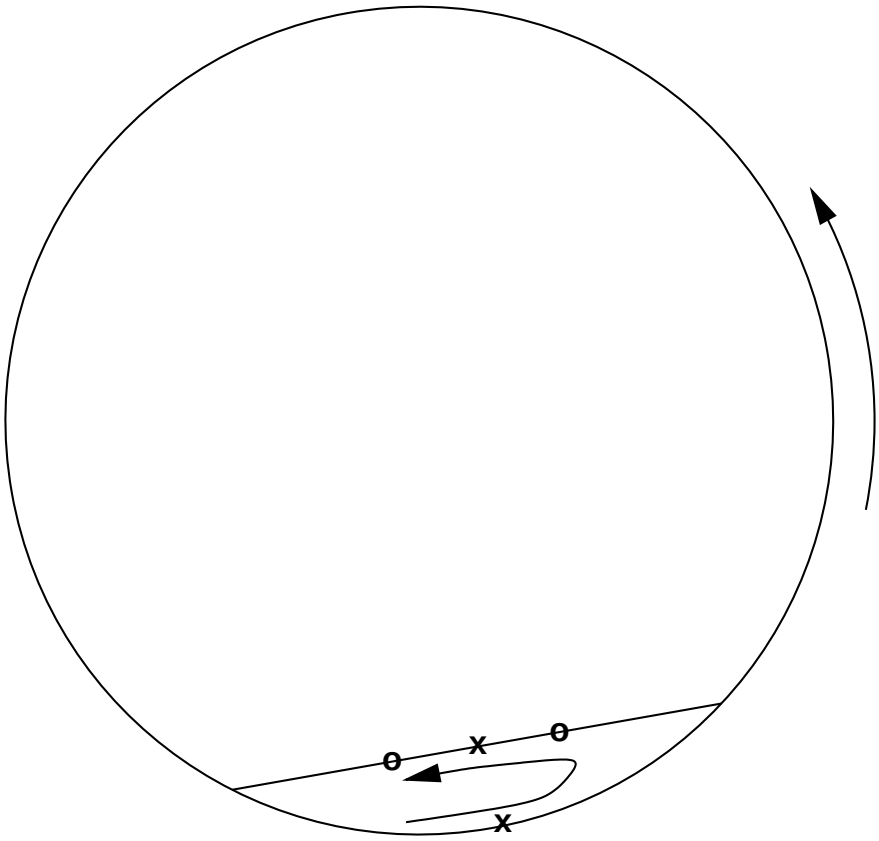


Figure 11: Schematic of rotary kiln

There is also one significant disadvantage: hyperbolic mesh generators are impractical for internal flows.

Good names to look for when exploring the literature on this include Joe Steger and William Chan.

3.2 Multi-block Patched and Chimera Meshes

For more complex geometries, single structured meshes are no longer appropriate. There are two common approaches to this problem, patches meshes and chimera meshes.

Patched meshes are created by decomposing the computational domain into a set of *blocks*, as shown schematically in Figure 14; the geometric boundaries are shown with heavy lines and the block boundaries with thin lines. The blocks can be meshed independently. Transfer

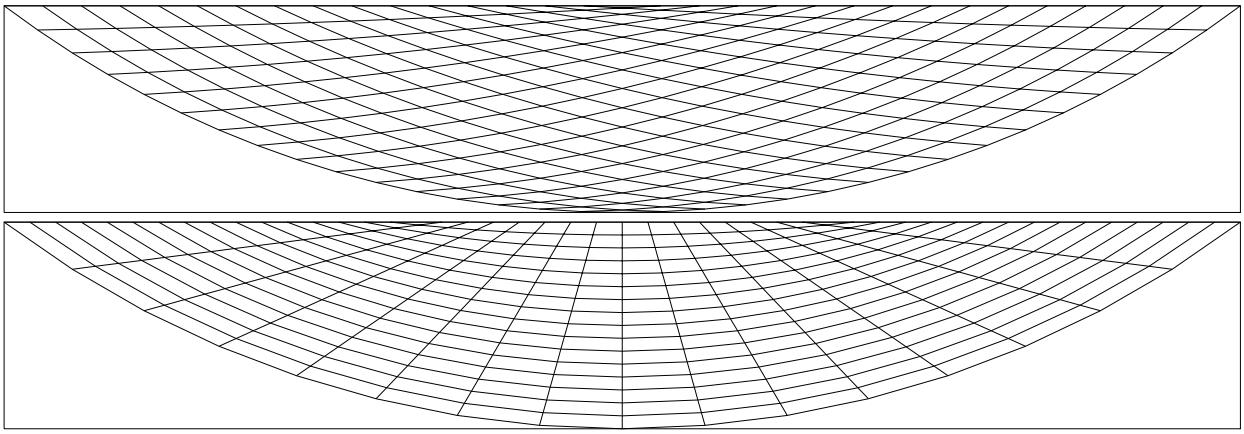


Figure 12: Kiln bed meshes using transfinite interpolation

of information between blocks is done through boundary conditions.

Chimera mesh systems are sets of overlapping meshes. In Figure 15, a single background mesh can be generated for the whole interior of the duct and local meshes generated around each cylinder. Inside the smaller meshes, the solution is calculated at all points, with some boundary conditions coming by interpolating data from the global mesh. The solution on the global mesh is calculated outside the sub-meshes and also in a small region of overlap with each submesh; mesh points that are farther into the sub-mesh than the overlap region are ignored. Boundary conditions around such 'holes' in the global mesh come from data interpolation from the sub-meshes.

References

[1] Thompson, Joe F., Z.U.A. Warsi, and C. Wayne Mastin. *Numerical Grid Generation: Foundations and Applications.* Elsevier, 1985.

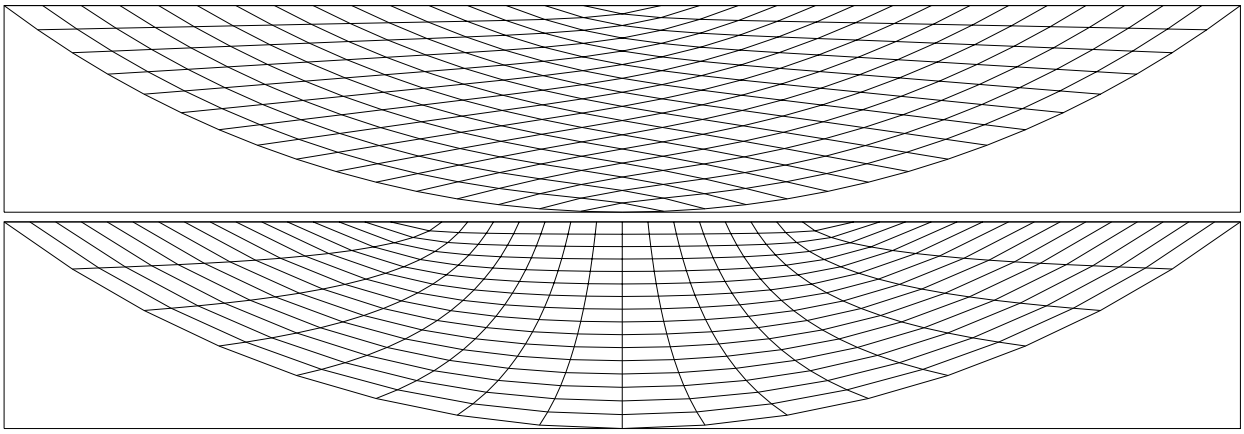


Figure 13: Kiln bed meshes using elliptical meshing

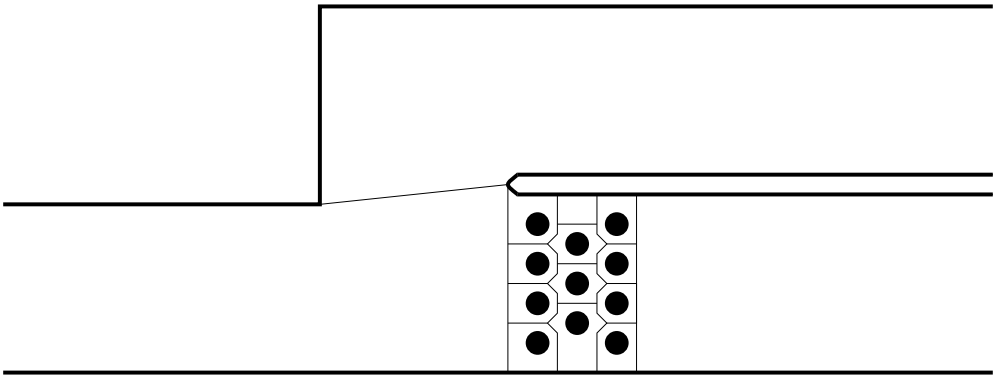


Figure 14: Block decomposition for branched duct.

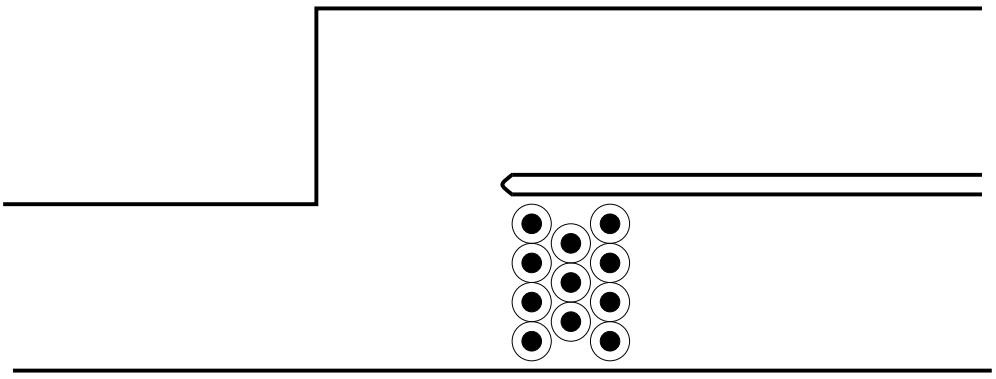


Figure 15: Chimera mesh boundaries for branched duct.